

Correction et terminaison des algorithmes

1. Pourquoi prouver un algorithme ?

Écrire un algorithme qui *semble* fonctionner sur quelques exemples ne garantit pas son absolue correction. Deux questions fondamentales se posent :

- **L'algorithme termine-t-il toujours ?** « Terminer » signifie ici que son déroulement a un terme : un algorithme qui boucle indéfiniment est inutilisable.
- **Produit-il toujours le bon résultat ?** Un algorithme qui termine mais donne parfois une mauvaise réponse est dangereux.

On parle respectivement de **terminaison** et de **correction**. Et il faut être en mesure d'apporter la preuve de chacune.

Remarque : pour la culture ▼

La preuve formelle est coûteuse en temps et en expertise : on ne la pratique que là où le coût d'un bug dépasse *largement* celui de la preuve. Pour les autres logiciels, on se contente de *tests*, qui ne peuvent que rarement couvrir *tous* les cas possibles. C'est précisément l'intérêt de la notion de preuve : un test montre qu'un algorithme fonctionne dans ce cas précis ; une preuve montre qu'il fonctionne dans *tous les cas*.

Voici quelques exemples de codes prouvés

- Ligne 14 (automatique) du métro de Paris : le logiciel embarqué du métro sans conducteur a été prouvé.
- Airbus : les systèmes de commande de vol sont développés selon une norme qui exige une vérification exhaustive du code. Certains composants critiques (comme la gestion des surfaces de vol) font l'objet de preuves formelles partielles.
- Centrales nucléaires : les systèmes d'arrêt d'urgence (SCRAM) sont soumis à des vérifications formelles très strictes.
- Carte à puce : les algorithmes cryptographiques des cartes bancaires ont été prouvés formellement (le CNRS et l'INRIA ont contribué à ces travaux).

2. Terminaison : le variant de boucle

2.1. Définition

Un **variant de boucle** est une expression (souvent évaluée en un nombre entier) qui vérifie deux conditions :

- elle **diminue strictement** à chaque itération de la boucle ;

- elle est **toujours positive ou nulle**.

Si une telle expression existe, la boucle *ne peut pas* s'exécuter indéfiniment : elle se termine nécessairement.

2.2. Exemple : recherche dichotomique

```
def dichotomie(liste, cible):
    gauche, droite = 0, len(liste) - 1
    while gauche <= droite:
        milieu = (gauche + droite) // 2
        if liste[milieu] == cible:
            return milieu
        elif liste[milieu] < cible:
            gauche = milieu + 1
        else:
            droite = milieu - 1
    return -1
```

Variant : `droite - gauche + 1` (la taille de la portion de liste encore examinée).

- À chaque itération, cette taille est au moins divisée par deux.
- Elle est toujours ≥ 0 .
- La boucle doit donc se terminer.

2.3. Exemple : tri par sélection

```
for i in range(len(liste)):           # boucle bornée : terminaison évidente
    ...
```

Les boucles `for` sont **naturellement bornées** : leur terminaison est immédiate (le nombre d'itérations maximum est connu à l'avance). La notion de variant n'est utile que pour les boucles `while`.

3. Correction : l'invariant de boucle

3.1. Définition

Un **invariant de boucle** est une propriété qui vérifie trois conditions :

- elle est **vraie avant d'entrer dans la boucle** (initialisation) ;
- si elle est vraie en *début* d'une itération, elle est **encore vraie en fin de cette itération** (conservation) ;
- elle implique, **à la sortie de la boucle**, que l'algorithme a produit le bon résultat (conclusion).

3.2. Exemple : tri par insertion

Le code suivant trie par insertion une liste par ordre croissant. **Rappel :** l'idée est de faire glisser une valeur à sa « bonne place » dans la liste, en faisant en sorte d'augmenter à chaque étape d'une unité le nombre de valeurs triées en début de liste. On réalise cela en « extrayant » la valeur *sélectionnée* de la liste, et en « faisant glisser vers la droite » les valeurs jusqu'à libérer *la bonne* place pour la valeur extraite.

```
def tri_insertion(liste):
    for i in range(1, len(liste)):
        cle = liste[i]
        j = i - 1
        while j >= 0 and liste[j] > cle:
            liste[j + 1] = liste[j]
            j -= 1
        liste[j + 1] = cle
```

Remarque : *slicing*1 ▼

Dans la suite, on utilise par commodité la notation Python pour extraire une portion d'une `liste`. Même si elle est *officiellement* hors programme, elle simplifie l'écriture, ici.

Ainsi, lorsque `n` est une variable préalablement déclarée et vérifiant `n < len(liste)`, `liste[1:n]` désigne une *nouvelle* liste, composée des éléments de `liste` situés aux positions `1`, `2`, ... `n-1`.

Invariant : « À la fin de l'itération `i`, le sous-tableau `liste[0:i+1]` est trié par ordre croissant. »

- **Initialisation :** avant la boucle (`i` non encore défini), `liste[0:1]` contient un seul élément : il est trivialement trié.
- **Conservation :** à l'itération `i`, on insère `liste[i]` à sa *bonne place* dans `liste[0:i]` (qui est déjà trié). Donc à la fin de l'itération, `liste[0:i+1]` est bien trié.
- **Conclusion :** à la fin de la boucle (`i = n-1`, où `n = len(liste)`), `liste[0:n]` — c'est-à-dire la liste entière — est triée.

3.3. Exemple : tri par sélection

Le code suivant trie par sélection une liste par ordre croissant. **Rappel :** le principe consiste à chercher la plus petite valeur de la liste et la mettre en position 0 ; puis chercher la deuxième plus petite valeur et la mettre en position 1 ; etc.

```
def tri_selection(liste):
    for i in range(len(liste)):
        min_idx = i
        for j in range(i + 1, len(liste)):
            if liste[j] < liste[min_idx]:
                min_idx = j
        liste[i], liste[min_idx] = liste[min_idx], liste[i]
```

Invariant : « À la fin de l'itération i , `liste[0:i]` contient les i plus petits éléments de la liste, triés par ordre croissant. »

- **Initialisation :** avant la boucle, `liste[0:0]` est vide : la propriété est vraie par *vacuité*.
- **Conservation :** à l'itération i , on place le minimum de `liste[i:]` en position i : la propriété est conservée.
- **Conclusion :** à la fin, toute la liste est triée.

 Remarque : *slicing* 2 ▼

En Python, `liste[i:]` crée une *nouvelle* liste comportant les éléments de `liste` de la position i à la dernière position inclus.

 Remarque : plus de rigueur ▼

À la fin de la boucle, i a pris sa dernière valeur $n-1$, donc l'invariant garantit que `liste[0:n-1]` contient les $n-1$ plus petits éléments triés. Cela implique que le dernier élément, nécessairement le plus grand, est bien en position $n-1$. Ainsi, la liste entière est bien triée.

4. Pour la culture : preuve par récurrence

La preuve formelle d'un invariant repose sur un **raisonnement par récurrence** :

- **Cas de base :** la propriété est vraie au rang 0 (avant la boucle).
- **Hérédité :** si elle est vraie au rang i , alors elle est vraie au rang $i+1$.
- **Conclusion :** elle est donc vraie pour tout rang, et en particulier à la sortie.

5. Attendu en NSI

Le schéma exposé à la section précédente correspond à celui de la démonstration par récurrence, au programme de l'enseignement de spécialité de mathématiques, en terminale. C'est donc hors spectre en NSI.

Dans une épreuve de baccalauréat de NSI, il est peu probable que *trouver* un variant ou un invariant soit demandé. Il est plus plausible qu'il soit fourni : il suffira alors de justifier que les trois étapes sont bien vérifiées (initialisation, conservation, conclusion).